

5位Solution

Rank	5
Team	Kinosuke
Author	Kinosuke
Topic ID	671211
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/671211>

Retrieved with Kaggle CLI on 2026-07-18.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

このコンペを開催してくださったhostに感謝します。分かりやすい画像taskで、参加を心から楽しみました。

概要

DINOv3のdense featureは極めて強力でした。私のapproachは、pre-trained weightを可能な限り保ちながら、そのfeatureを効果的に活用することへ集中しました。

Global feature（CLS）を使うとDINOv3の強力なlocal separation能力が失われます。そこで、できるだけ segmentation taskに近いmodelを設計しました。

- DINOv3のdense featureを使うend-to-end training
- 最大50%までのgradual unfreeze。full fine-tuningは行わない
- 2000×1000画像を半分に分け、labelも半分にしてtraining dataを2倍化
- shake robustnessのため、Large/Huge+、複数解像度、2 seedによる6 model ensemble
- 9時間制約へ収めるinference optimization（T4×2 parallel inference、FP16）

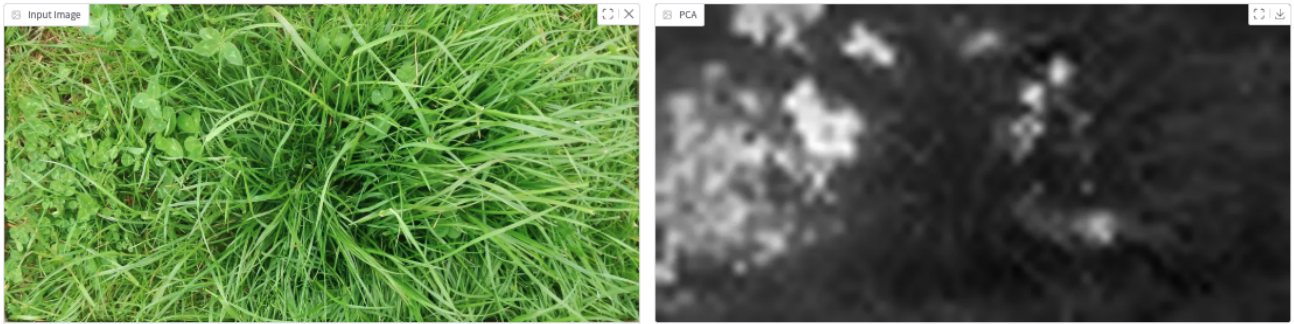
Model	Resolution	CV (OOF R ²)	Public	Private
DINOv3 Huge+ (child-exp012)	672	0.809	0.75	0.65
DINOv3 Huge+ (child-exp017)	768	0.816	0.75	0.65
DINOv3 Huge+ (child-exp026)	672 + PatchDropout	0.811	0.75	0.65
DINOv3 Huge+ (child-exp032)	864	0.814	0.75	0.65
DINOv3 Large (child-exp020)	864	0.818	0.73	0.66
DINOv3 Large (child-exp037)	960	0.819	0.74	0.65

Model

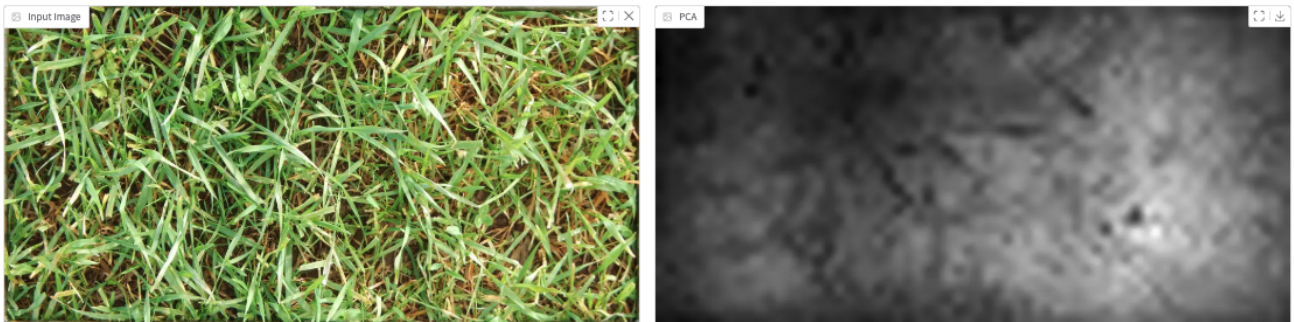
なぜdensity estimationか

DINOv3 featureをPCAで可視化すると、zero-shotでもlocal separationに優れていることが分かります。Green、Clover、Dead（一部）、Soilを明確に区別できます。

Cloverへfocus：



Deadへfocus :



注：分離が視覚的に分かりやすいsampleを意図的に選んでいます。すべてのsampleがこれほどきれいに分かれるわけではありません。

この優れたlocal separation能力があるにもかかわらず、CLS tokenへheadを付けるとspatial informationを捨てることになり、もったいないと感じました。そこでdensity estimation approachを採用しました。各patch tokenからlocal biomass densityを予測し、画像全体でsum (integrate) します。

Architecture: Density-only head

```

Input image (1000x1000, one half of left-right split)
  | resize
Input (672x672x3)
  |
DINOv3 ViT-Huge+ backbone (frozen -> gradual unfreeze)
  |
Patch tokens [B, 1764, 1280] (42x42 patches, 1280-dim each)
  | reshape
[B, 1280, 42, 42]
  |
1x1 Conv (1280 -> 5)  <- Only 6,405 learnable parameters here
  |
Softplus (non-negativity constraint: biomass density is physically non-negative)
  |
Density map [B, 5, 42, 42] (local density for 5 targets per patch)
  |
Sum (aggregate all patches)
  |
Prediction [B, 5] = [Green, Dead, Clover, GDM, Total]

```

重要な点は次の通りです。

- **CLS tokenは使いません。** 予測はpatch tokenだけにに基づくDensity-onlyです。

- **1×1 Conv**で各patchの1280-dimensional featureを5つのbiomass densityへ線形変換します（spatial convolutionなし）。
- **Softplus**で非負性を課し、各patchは「この領域にどれほどbiomassがあるか」を表します。
- 全patchの**sum**は `biomass = density × area` という物理関係をencodeします。

学習を収束させるのは困難でした。隣接patchの関係を学ぶためConv3×3も試しましたが、learning rateをどう調整してもNaN lossを解消できませんでした。より単純なConv1×1だけで学習が安定しました。また、validation scoreが安定するか最初の5〜8 epochsを監視し、収束しないrunは手動で止めました。

Resolution

resolutionは極めて重要でした。高解像度にするほどCVとLBが一貫して改善しました。224から始め、最終的に960まで上げました。

高解像度はtraining timeとVRAMを多く必要とするため、実験は256で開始して512へ上げ、コンペ最後の3週間で768へ進みました。最終modelはLarge 960px、Huge+ 864pxです。Huge+ 864pxの5-fold学習は、A100 80GBでbatch size 8を使って20時間かかりました。

Training strategy

Gradual unfreezeが鍵でした。DINOv3のように強いpre-trained weightを持つmodelを最初からfull fine-tuningすると、価値あるpre-trained representationを壊してしまいます。

Gradual unfreeze

- **Epoch 1〜5:** 高いlearning rateでheadだけを学習し、backboneをfreeze。
- **Epoch 6〜40:** headに近いlayerから徐々にunfreeze。

例えばDINOv3 Largeは24 Transformer blockを持ちます。`max_unfreeze_ratio=50%` では、epoch 40までに最大12 blockをunfreezeします。

重要な設計判断は、**learning rate schedulerを使わない**ことです。多くのschedulerはwarmup後に徐々にdecayするため、後半でunfreezeされたlayerへ十分な学習量が届きません。代わりにbackbone learning rateを線形に増やしました。

- 開始 (epoch 5) : 1e-5
- 終了 (epoch 40) : 3e-5

Data split（左右半分）

多くのpublic notebookは2000×1000画像を左右へ分け、最終layerの出力を連結してheadへ入力しました。

私の方法は異なり、画像を左右へ分け、**labelも半分にしました**。これによりtraining dataは実質2倍です。この方法はDINOv3、EVA02-CLIP、SigLIPのすべてで2-stream concatenationを一貫して上回りました。

Loss function

base lossはSmoothL1Lossで、5targetすべてを予測します。重要な追加要素は、物理関係を課す**ConsistencyLoss**です。

- `Total = Green + Dead + Clover`
- `GDM = Green + Clover`

当初は独立な3target（Green、Dead、Clover）だけを予測し、残りを導出する方が効率的だと考えました。しかしCloverとGreenの間にlabel noiseがあるように見えたため、5つすべてを予測しconsistency constraintを加える方法が有効なregularizationになりました。

CV strategy

[このDiscussion](#)のgroup-based approachを採用すると、CV-LB相関が安定しました。

CloverとDeadは予測が難しいため、stratificationで各foldへ均等に分配しました。

```
df_wide['clover_dead_presence'] = (  
    (df_wide['Dry_Clover_g'] > 0).astype(int).astype(str) + '_' +  
    (df_wide['Dry_Dead_g'] > 0).astype(int).astype(str)  
)
```

これをStratifiedGroupKFoldのstratification変数に使い、CloverまたはDeadが0のsampleをfold間で均等にしました。これらは0のsampleが多く、単純random splitでは特定foldが偏る可能性があります。

値	Clover	Dead	意味
0_0	なし (=0)	なし (=0)	両方0
0_1	なし (=0)	あり (>0)	Deadのみ
1_0	あり (>0)	なし (=0)	Cloverのみ
1_1	あり (>0)	あり (>0)	両方あり

これによりCV-LB相関は非常に安定しました。

This scatter plot displays the relationship between the Out-of-Fold (OOF) coefficient of determination (R^2) and the Public Loss (LB) for two datasets: EXP060 (blue circles) and EXP113 (red circles). The x-axis represents OOF R^2 (CV), ranging from 0.75 to 0.83. The y-axis represents Public LB, ranging from 0.65 to 0.75. A dashed black line indicates the trend, with a correlation coefficient $r = 0.831$. A yellow star marks the best-performing model from EXP113, E113-017, which has a Public LB of 0.75. The legend identifies the data series and the trend line.

Dataset	OOF R^2 (CV)	Public LB
EXP060 (n=32)	0.755	0.680
EXP060 (n=32)	0.760	0.670
EXP060 (n=32)	0.765	0.685
EXP060 (n=32)	0.770	0.660
EXP060 (n=32)	0.770	0.660
EXP060 (n=32)	0.775	0.670
EXP060 (n=32)	0.775	0.670
EXP060 (n=32)	0.780	0.670
EXP060 (n=32)	0.780	0.670
EXP060 (n=32)	0.785	0.670
EXP060 (n=32)	0.790	0.670
EXP060 (n=32)	0.795	0.670
EXP060 (n=32)	0.800	0.670
EXP060 (n=32)	0.805	0.670
EXP060 (n=32)	0.760	0.690
EXP060 (n=32)	0.780	0.690
EXP060 (n=32)	0.790	0.710
EXP060 (n=32)	0.795	0.710
EXP060 (n=32)	0.800	0.710
EXP060 (n=32)	0.805	0.710
EXP060 (n=32)	0.770	0.700
EXP060 (n=32)	0.775	0.710
EXP060 (n=32)	0.785	0.710
EXP060 (n=32)	0.790	0.710
EXP060 (n=32)	0.795	0.710
EXP060 (n=32)	0.800	0.710
EXP060 (n=32)	0.805	0.710
EXP060 (n=32)	0.790	0.720
EXP060 (n=32)	0.800	0.720
EXP060 (n=32)	0.805	0.720
EXP060 (n=32)	0.795	0.730
EXP060 (n=32)	0.800	0.730
EXP060 (n=32)	0.770	0.700
EXP060 (n=32)	0.775	0.710
EXP060 (n=32)	0.780	0.710
EXP060 (n=32)	0.785	0.710
EXP060 (n=32)	0.790	0.710
EXP060 (n=32)	0.795	0.710
EXP060 (n=32)	0.800	0.710
EXP060 (n=32)	0.805	0.710
EXP060 (n=32)	0.790	0.720
EXP060 (n=32)	0.800	0.720
EXP060 (n=32)	0.805	0.720
EXP060 (n=32)	0.795	0.730
EXP060 (n=32)	0.800	0.730
EXP060 (n=32)	0.805	0.730
EXP060 (n=32)	0.790	0.740
EXP060 (n=32)	0.800	0.740
EXP060 (n=32)	0.805	0.740
EXP060 (n=32)	0.795	0.750
EXP060 (n=32)	0.800	0.750
EXP060 (n=32)	0.805	0.750
EXP113 (n=18)	0.800	0.750
EXP113 (n=18)	0.805	0.750
EXP113 (n=18)	0.810	0.750
EXP113 (n=18)	0.815	0.750
EXP113 (n=18)	0.800	0.730
EXP113 (n=18)	0.805	0.730
EXP113 (n=18)	0.810	0.730
EXP113 (n=18)	0.815	0.730
EXP113 (n=18)	0.800	0.720
EXP113 (n=18)	0.805	0.720
EXP113 (n=18)	0.810	0.720
EXP113 (n=18)	0.815	0.720
EXP113 (n=18)	0.800	0.710
EXP113 (n=18)	0.805	0.710
EXP113 (n=18)	0.810	0.710
EXP113 (n=18)	0.815	0.710
EXP113 (n=18)	0.800	0.700
EXP113 (n=18)	0.805	0.700
EXP113 (n=18)	0.810	0.700
EXP113 (n=18)	0.815	0.700
EXP113 (n=18)	0.800	0.690
EXP113 (n=18)	0.805	0.690
EXP113 (n=18)	0.810	0.690
EXP113 (n=18)	0.815	0.690
EXP113 (n=18)	0.800	0.680
EXP113 (n=18)	0.805	0.680
EXP113 (n=18)	0.810	0.680
EXP113 (n=18)	0.815	0.680
EXP113 (n=18)	0.800	0.670
EXP113 (n=18)	0.805	0.670
EXP113 (n=18)	0.810	0.670
EXP113 (n=18)	0.815	0.670
EXP113 (n=18)	0.800	0.660
EXP113 (n=18)	0.805	0.660
EXP113 (n=18)	0.810	0.660
EXP113 (n=18)	0.815	0.660
EXP113 (n=18)	0.800	0.650
EXP113 (n=18)	0.805	0.650
EXP113 (n=18)	0.810	0.650
EXP113 (n=18)	0.815	0.650

使用したaugmentationは次の通りです。

```
HorizontalFlip (p=0.5)
VerticalFlip (p=0.5)
RandomRotate90 (p=0.5)
Rotate (limit=10, p=0.3)
RandomResizedCrop (scale=0.85-1.0, ratio=0.95-1.05, p=0.5)
ColorJitter (brightness=0.3, contrast=0.3, saturation=0.3, hue=0.15, p=0.7)
RandomGamma (gamma_limit=80-120, p=0.3)
RandomBrightnessContrast (brightness=0.25, contrast=0.25, p=0.5)
GaussianBlur (blur_limit=3-5, p=0.2)
RandomShadow (p=0.1)
RandomToneCurve (p=0.2)
```

`RandomResizedCrop` は見えるgrass量を変えるため、当初は使うべきでないと思いました。しかし実際には精度が上がりました。撮影者によってcamera-to-ground distanceが変わり、このaugmentationがその変動へのrobustnessを高めた可能性があります。

強すぎるaugmentationはperformanceを悪化させました。

- hostのpaperにはさまざまなcamera typeで撮影したと書かれていたため、多様なnoise augmentationを加えましたが、やり過ぎると悪化しました。
- `RandomGridShuffle` も効果がありませんでした。

Shake対策

2-seed ensemble

小標本のregression taskなので、大きなshakeを予想しました。Feedback PrizeコンペのPsiのDiscussionに着想を得て、2 seedで学習・評価し、modelが本当に一般化しているか確認するとともにrobustnessを高めました。

Model比較

コンペ序盤から参加し、SigLIP→EVA02-CLIP→DINOv3とmodelを段階的にupgradeしました。私は通常CLIP-based modelを好みますが、このコンペではDINOv3が圧倒的に優れていました。

下表のmodelは同じtraining pipeline (augmentation、learning rate、gradual unfreeze) を使い、VRAMへ合わせてbatch sizeだけを調整しています。

Model	CV	Public	Private
SigLIP	0.75	0.66	0.55
EVA02-CLIP	0.75	0.69	0.584
DINOv3 [Global Feature]	0.79	0.73	0.64
DINOv3 [Dense Feature]	0.81	0.75	0.66

うまくいかなかったこと

- **TTA** — 改善なし
- **Sliding Window Inference** — 左右分割の代わりに500pxずつslideして4 passしたが効果なし
- **Data cleaning** — WA/2015-8-21 のlabelは特にnoisyだった。手修正でCVは改善したがLBは悪化した。testにも同じlabeling errorがある可能性が高い
- **Auxiliary loss** — Height、Species、Seasonをauxiliary targetにしても効果なし
- **Depth Anything** — depth mapはtrain.csvのHeightと強く相関した。patchごとのheightを入れるためdensity mapへdepthを掛けたが改善なし
- **EMA** — 効果なし。ただし実装が間違っていた可能性があり、深く調査していない